

建表SQL

团队：第67组 日期：2026/5/7

建表SQL语句

目前暂时根据类图与E-R图，通过以下SQL语句建立表

-- 1. 用户基础表（实现类表继承，存储通用凭证）

```
CREATE TABLE `users` (  
  `user_id` BIGINT PRIMARY KEY AUTO_INCREMENT,  
  `student_no` VARCHAR(20) NOT NULL UNIQUE COMMENT '学号，唯一标识',  
  `password_hash` VARCHAR(255) NOT NULL COMMENT 'Argon2或BCrypt加密后的哈希值',  
  `status` TINYINT DEFAULT 1 COMMENT '1:正常, 0:禁用',  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  INDEX `idx_student_no` (`student_no`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 2. 学生扩展表（存储业务属性）

```
CREATE TABLE `student_users` (  
  `student_user_id` BIGINT PRIMARY KEY,  
  `user_id` BIGINT NOT NULL,  
  `nickname` VARCHAR(50),  
  `college` VARCHAR(100),  
  `contact_info_enc` VARBINARY(255) COMMENT '加密存储的联系方式',  
  `credit_score` DECIMAL(5,2) DEFAULT 100.00,  
  `current_role` ENUM('DEMANDER', 'SERVER') DEFAULT 'DEMANDER',  
  FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`),  
  INDEX `idx_user_id` (`user_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 3. 需求发布表

```
CREATE TABLE `help_requests` (  
  `request_id` BIGINT PRIMARY KEY AUTO_INCREMENT,  
  `publisher_id` BIGINT NOT NULL COMMENT '对应student_user_id',  
  `category_id` INT NOT NULL COMMENT '分类配置表ID',  
  `title` VARCHAR(100) NOT NULL,  
  `description` TEXT NOT NULL,  
  `reward_amount` DECIMAL(10,2) DEFAULT 0.00,  
  `deadline` DATETIME NOT NULL,  
  `status` TINYINT DEFAULT 0 COMMENT '0:待审核, 1:招募中, 2:已匹配, 3:已完成, 4:已  
关闭',  
  `is_anonymous` BOOLEAN DEFAULT FALSE,  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (`publisher_id`) REFERENCES `student_users` (`student_user_id`),  
  INDEX `idx_status_category` (`status`, `category_id`),  
  INDEX `idx_deadline` (`deadline`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- 4. 服务订单表
CREATE TABLE `service_orders` (
  `order_id` BIGINT PRIMARY KEY AUTO_INCREMENT,
  `request_id` BIGINT NOT NULL UNIQUE,
  `demander_id` BIGINT NOT NULL,
  `server_id` BIGINT NOT NULL,
  `current_state` VARCHAR(50) NOT NULL COMMENT '状态模式对应的状态类名',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `completed_at` DATETIME,
  `review_score` TINYINT,
  `review_comment` TEXT,
  `has_complaint` BOOLEAN DEFAULT FALSE,
  `complaint_reason` TEXT,
  FOREIGN KEY (`request_id`) REFERENCES `help_requests`(`request_id`),
  FOREIGN KEY (`demander_id`) REFERENCES `student_users`(`student_user_id`),
  FOREIGN KEY (`server_id`) REFERENCES `student_users`(`student_user_id`),
  INDEX `idx_demander` (`demander_id`),
  INDEX `idx_server` (`server_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

索引设计说明

1. 用户基础表 (users)

索引名	索引类型	字段	设计目的
PRIMARY	主键索引	user_id	唯一标识每条用户记录，提高按ID查询效率
idx_student_no	普通索引	student_no	学号是唯一登录凭证，频繁用于登录验证和用户查找，加速查询

2. 学生扩展表 (student_users)

索引名	索引类型	字段	设计目的
PRIMARY	主键索引	student_user_id	唯一标识学生扩展记录
idx_user_id	普通索引	user_id	用于 users 与 student_users 的关联查询，如登录后获取学生完整信息

user_id 作为外键，可能频繁参与联表查询，建立索引可避免全表扫描。

3. 需求发布表 (help_requests)

索引名	索引类型	字段	设计目的
PRIMARY	主键索引	request_id	唯一标识每条需求
idx_status_category	复合索引	(status, category_id)	核心查询场景：按状态和分类筛选需求（如“招募中的编程类需求”），复合索引可同时过滤两个条件
idx_deadline	普通索引	deadline	用于定时任务扫描即将到期或已过期的需求，按截止时间排序和筛选

复合索引的字段顺序为 (status, category_id)，因为 status 的区分度更高（枚举值少但查询频率高），放在左侧可快速过滤。

4. 服务订单表 (service_orders)

索引名	索引类型	字段	设计目的
PRIMARY	主键索引	order_id	唯一标识每笔订单
idx_demander	普通索引	demander_id	需求方查询“我发布的需求对应的订单”
idx_server	普通索引	server_id	服务方查询“我接单的订单”

- request_id 已通过 UNIQUE 约束保证一对一关系，通常通过 request_id 反查订单，但考虑到业务中查看自己订单时需要需求方/服务方视角查询，单独建立两个外键索引。

数据项设计说明

1. 用户基础表 (users)

字段名	类型	约束	说明
user_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	用户唯一标识，内部使用，不对外暴露
student_no	VARCHAR(20)	NOT NULL, UNIQUE	学号，作为登录凭证，唯一标识学生身份
password_hash	VARCHAR(255)	NOT NULL	存储 Argon2/BCrypt 加密后的密码哈希值，长度预留足够空间
status	TINYINT	DEFAULT 1	账号状态：1-正常，0-禁用（冻结/毕业注销）
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	账号创建时间

字段名	类型	约束	说明
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	最后更新时间，用于审计追踪

设计说明：采用“类表继承”模式，本表存储所有用户类型的通用凭证，扩展表存储学生特有属性，便于后续扩展管理员角色。

2. 学生扩展表（student_users）

字段名	类型	约束	说明
student_user_id	BIGINT	PRIMARY KEY	学生扩展记录ID，与 user_id 独立，便于分表扩展
user_id	BIGINT	NOT NULL, FOREIGN KEY	关联 users.user_id，1对1关系
nickname	VARCHAR(50)		用户昵称，可自定义，允许为空
college	VARCHAR(100)		所在院系，用于筛选同院系互助场景
contact_info_enc	VARBINARY(255)		联系方式（手机号/邮箱等），使用 AES 等算法加密存储，保护隐私
credit_score	DECIMAL(5,2)	DEFAULT 100.00	信用分，范围 0~100，用于信用体系建设（违约扣分、优质履约加分）
current_role	ENUM	DEFAULT 'DEMANDER'	当前角色：DEMANDER-需求方，SERVER-服务方，用户可切换

设计说明：

- 联系方式采用 VARBINARY 而非 VARCHAR，明确表示存储的是加密后的二进制数据。
- 信用分与角色分离，支持用户在不同场景下切换身份。

3. 需求发布表（help_requests）

字段名	类型	约束	说明
request_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	需求唯一标识
publisher_id	BIGINT	NOT NULL, FOREIGN KEY	发布者ID，关联 student_users.student_user_id
category_id	INT	NOT NULL	需求分类ID，关联分类配置表（待补充），如编程、设计、跑腿等
title	VARCHAR(100)	NOT NULL	需求标题，简洁概括
description	TEXT	NOT NULL	详细描述，支持富文本或纯文本

字段名	类型	约束	说明
reward_amount	DECIMAL(10,2)	DEFAULT 0.00	悬赏金额，0表示无偿互助
deadline	DATETIME	NOT NULL	招募截止时间，超时自动关闭
status	TINYINT	DEFAULT 0	状态：0-待审核，1-招募中，2-已匹配，3-已完成，4-已关闭
is_anonymous	BOOLEAN	DEFAULT FALSE	是否匿名发布，保护隐私
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	发布时间

状态流转说明：

- 待审核(0) → 招募中(1) → 已匹配(2) → 已完成(3)
- 任意非终态均可进入已关闭(4)

4. 服务订单表（service_orders）

字段名	类型	约束	说明
order_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	订单唯一标识
request_id	BIGINT	NOT NULL, UNIQUE, FOREIGN KEY	关联的原始需求ID，确保一个需求只产生一个订单
demander_id	BIGINT	NOT NULL, FOREIGN KEY	需求方ID（等于 help_requests.publisher_id）
server_id	BIGINT	NOT NULL, FOREIGN KEY	接单服务方ID
current_state	VARCHAR(50)	NOT NULL	状态模式中的状态类名，如 PendingState、InProgressState 等
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	订单创建时间（匹配成功时间）
completed_at	DATETIME		订单完成时间
review_score	TINYINT		评价分数，1~5星或1~10分，由需求方填写
review_comment	TEXT		评价文本
has_complaint	BOOLEAN	DEFAULT FALSE	是否发起过投诉
complaint_reason	TEXT		投诉原因（若有）

设计说明：

- current_state 使用 VARCHAR(50) 存储状态类名，配合后端状态模式实现灵活的状态机逻辑。

- `review_score` 与 `review_comment` 允许为空，表示尚未评价。
- 投诉字段与主订单表合一，简化设计；若投诉逻辑复杂后可拆分为独立的 `complaints` 表。